

Text as Data for Economic Research

Machine Learning with Text

Class 2. Supervised Prediction and Unsupervised Discovery
June 8, 2026

Tiziano Rotesi

University of Turin

Plan for Today

- **Framing:** machine learning as a *measurement* workflow
- **Supervised:** label $\rightarrow$ train $\rightarrow$ predict the rest
 - the model ladder, evaluation, regularization, scaling
- **Unsupervised:** the vector-space model, clustering, dimension reduction
 - discovering structure — and why it is exploration, not measurement
- Where large language models fit — and why today's tools still matter

Framing: ML as Measurement

Two Ways to Get a Measure from Text

In Class 1 we **wrote a dictionary by hand** and counted matches.

Today we let **machine learning** do the work:

- **Supervised** — we *learn* the dictionary from labeled examples instead of writing it.
- **Unsupervised** — we *discover* categories no one wrote down.

Same goal as Class 1: turn a corpus into a variable we can put in a regression.

The Invariant: a Measurement Workflow

Whatever the model, the recipe is the same:

1. **Label** a subset (by hand, or from an existing field)
2. **Train** a map from text features to the label
3. **Evaluate** out of sample
4. **Apply** to the rest — scale the measure cheaply
5. **Validate** against human judgment; treat the output as data with error

This workflow does not change when we swap a logistic regression for a large language model (Class 4). The estimator is a vehicle; the workflow is the lesson.

Why Start With the Transparent Models

Naive Bayes and logistic regression are **fully legible**: we can write the loss, the gradient, and the prediction error in closed form.

- You can **see where the prediction error comes from** — exactly the object we need for the **econometrics of a generated variable** (Class 5).
- That transparency *fades up the ladder* — a tree **ensemble** is already opaque, a Transformer more so. Here the error is **visible** — the right place to build the intuition we formalize in Class 5.

The Class 5 correction is **model-agnostic** — it needs a validation sample, not a transparent model — so the same logic carries to the LLM: same task, harder inference.

Supervised Prediction

How Economists Use ML on Text

Two broad uses — the two halves of today:

- **Predict / measure a variable** — map text $\rightarrow$ a label or score per document, then use it as data in a regression.
 - *The dominant use.* Sentiment of reviews (today's notebook); partisanship of corporate tweets (Kempf & Cassidy 2025).
 - *A variant — group differences:* the object is not each label but **how much two groups differ** in language, read off how well a model separates them (Gentzkow, Shapiro & Taddy 2019). We return to this.
- **Discover structure** — no labels: cluster or topic-model a corpus to see what is in it. Exploration, not measurement.

Predict is the **supervised** half of today; discover the **unsupervised** half (Ash & Hansen 2023).

Supervised Learning

Classification as a supervised problem:

- You know how to label *some* of the documents.
- Labeling *every* document is infeasible:
 - the corpus is too large to read by hand;
 - you also want to predict *future* documents.

Build a model that *learns* to label documents using the labels you provide.

Goal: Learn a Mapping Function

Learn the **mapping** from text to the quantity of interest, using a sample of coded documents.

- We have set up a **prediction problem**.
- The target sets the *flavour*: a **category** gives **classification** (most of today); a **number** gives **regression** (e.g. OLS, next).
Same four ingredients — only the loss changes.

Familiar territory for economists — but the regressors are now thousands of sparse word counts.

Four Ingredients of a Supervised Model

- **Features** — how text becomes input $\rightarrow \mathbf{x}_i$ (tf-idf, counts, n -grams)

Four Ingredients of a Supervised Model

- **Features** — how text becomes input $\rightarrow \mathbf{x}_i$ (tf-idf, counts, n -grams)
- **Statistical model** — text and parameters to a predicted label $\rightarrow f(\mathbf{x}_i, \mu)$

Four Ingredients of a Supervised Model

- **Features** — how text becomes input $\rightarrow \mathbf{x}_i$ (tf-idf, counts, n -grams)
- **Statistical model** — text and parameters to a predicted label $\rightarrow f(\mathbf{x}_i, \boldsymbol{\mu})$
- **Loss function** — distance between predicted and actual labels $\rightarrow L(f(\mathbf{x}_i, \boldsymbol{\mu}), y_i)$

Four Ingredients of a Supervised Model

- **Features** — how text becomes input $\rightarrow \mathbf{x}_i$ (tf-idf, counts, n -grams)
- **Statistical model** — text and parameters to a predicted label $\rightarrow f(\mathbf{x}_i, \boldsymbol{\mu})$
- **Loss function** — distance between predicted and actual labels $\rightarrow L(f(\mathbf{x}_i, \boldsymbol{\mu}), y_i)$
- **Regularization** — a penalty on parameter size to prevent overfitting $\rightarrow R(\boldsymbol{\mu}, \lambda)$

Every model today is a choice of these four pieces.

Training as an Optimization Problem

Objective: find parameters μ minimizing total penalized loss:

$$\arg \min_{\mu} \left(\sum_{i=1}^N L(f(\mathbf{x}_i, \mu), y_i) + R(\mu, \lambda) \right)$$

- A clear, measurable training objective.
- Lets us reuse proven optimization algorithms to find μ .

The Simplest Case You Already Know: OLS

Linear regression is machine learning with a specific choice of the four ingredients:

- **Features $\mathbf{X}$** , outcome $\mathbf{y}$;
- **Model:** $f(\mathbf{X}, \boldsymbol{\beta}) = \mathbf{X}\boldsymbol{\beta}$ — linear in the parameters;
- **Loss:** the sum of squared residuals;
- **Regularization:** none (for now).

Estimating the coefficients means solving

$$\hat{\boldsymbol{\beta}} = \arg \min_{\boldsymbol{\beta}} \sum_{i=1}^N (y_i - \mathbf{X}_i \boldsymbol{\beta})^2.$$

There are **two ways** to solve this — and the contrast is the template for every model today.

OLS, Two Ways: (1) the Closed Form

For OLS the minimization has an **analytical solution** — the familiar estimator:

$$\hat{\beta}_{OLS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

- One line of linear algebra; exact; nothing to tune.
- **But** it is special to squared-error loss with a linear model, and it needs $\mathbf{X}^\top \mathbf{X}$ to be invertible — which fails when features outnumber documents (the usual case for text).

Most models today have **no** closed form. We need a general-purpose solver.

OLS, Two Ways: (2) the Algorithmic Approach

Gradient descent approximates $\hat{\beta}$ by walking downhill on the loss:

1. start from a guess β_0 ;
 2. compute the gradient $\nabla L(\beta)$ — the direction of steepest *increase*;
 3. step the opposite way, $\beta_{t+1} = \beta_t - \alpha \nabla L(\beta_t)$, with learning rate α ;
 4. repeat until β stops moving.
- Converges to the global optimum whenever the loss is **convex**.
 - Needs no closed form — so the *same* solver fits logistic regression, tree boosting, and (Class 3) word embeddings.

Parameters vs. Hyperparameters

- **Parameters** — what the training step *solves for* by minimizing the (penalized) loss on the training data: the coefficients β , the word-given-class probabilities, a tree's split points. They are **identified by the data**.
- **Hyperparameters** — the knobs you set *before* training that govern **model capacity** and the **learning procedure** itself: the penalty strength λ , vocabulary size and n -gram range, tree depth, the learning rate, the number of CV folds.

Why not just fit the hyperparameters too? Pushed to minimize *training* loss they would drive it to zero by memorizing the sample. They are chosen instead by **out-of-sample** performance — the subject of the next slides.

Features and the Train/Test Split

- Represent each document with tf-idf (or counts), as in Class 1. The **vectorizer** is the map raw text $\rightarrow$ feature matrix: it tokenizes, builds the **vocabulary**, and fills in counts / tf-idf weights.
- It has **parameters estimated from data**, like the model itself: which terms become columns (the **vocabulary**) and the **idf weights** both depend on the corpus it sees. `fit` estimates them on a corpus; `transform` applies them — so it is part of the model, not neutral preprocessing.
- **Split before fitting.** Fit the vectorizer *and* the model on **train** only; transform the test set with the *same* vocabulary. Letting the test set shape the vocabulary leaks information and inflates accuracy — the most common mistake in applied ML.

Why Hold Out Data

A flexible model can fit the training set almost perfectly — and still fail on new documents (**overfitting**).

- Split into a **training set** ($\approx 80\%$) and a **test set** ($\approx 20\%$).
- The split is random; the test set proxies the population.
- The test set is touched *only once*, to report performance.

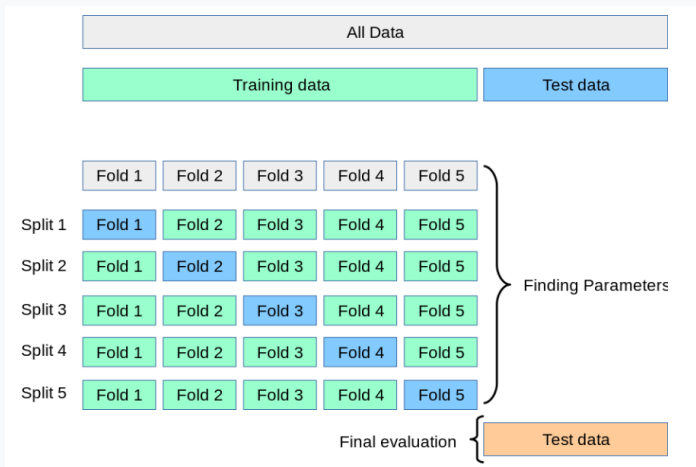
Choosing Hyperparameters: Cross-Validation

Hyperparameters must be chosen *without* peeking at the test set.

1. **Within the training set:** split repeatedly into fit/validation folds; for each candidate value, measure out-of-fold performance and keep the best.
2. Refit on the full training set with that value; report performance **once**, on the held-out test set.

Cross-validation is computationally costly, but it is what keeps the test set untouched — and the reported number genuinely out-of-sample.

Cross-Validation, Visually



Source: scikit-learn.

The Model Ladder

What Is a Classifier?

A **classifier** makes the map f concrete: it takes a document's features and returns a class.

$$f : \mathbf{x}_i \mapsto \hat{p}(k | \mathbf{x}_i) \xrightarrow{\text{decision rule}} \hat{y}_i \in \{1, \dots, K\}.$$

- It outputs a **probability over classes**; a **decision rule** (typically “pick the most likely class”) turns that into a label.
- Its parameters are fit by minimizing the penalized loss; it is judged **out of sample**.

The models below differ only in the *shape* of f and what they assume — not in this workflow.

A Menu of Models — Ordered by Assumptions

We climb a ladder of model *families*, from transparent to powerful:

1. **Naive Bayes** — a generative count model
2. **Logistic regression** — a linear discriminative model
3. **Tree ensembles** — random forests, gradient boosting
4. **Fine-tuned Transformers** — the frontier (Class 4)

Regularization is not a rung — it is a knob we can add to any of these to control overfitting.

Ash & Hansen (2023): the right rung depends on the *task* and the *corpus size*, not on reaching for the top — penalized linear models are often too limited for text, while ensembles capture the interactions that term counts carry.

Naive Bayes: the Generative Idea

Turn the question around. Instead of modelling the label given the words, model how a **class generates words**, then invert with Bayes' rule:

$$\underbrace{P(\text{class} \mid \text{words})}_{\text{what we want}} \propto \underbrace{P(\text{class})}_{\text{prior}} \cdot \underbrace{P(\text{words} \mid \text{class})}_{\text{generative model}}.$$

- Each class has its own word-probability vector — the **multinomial language model** of Class 1, now one per label.
- A 5★ review draws words from the “positive” vocabulary; a 1★ review from the “negative” one.

Naive Bayes: Why “Naive”

Computing $P(\text{words} \mid \text{class})$ exactly would need the joint distribution of *all* words at once — hopeless. The **naive** assumption: words are **conditionally independent** given the class, so the joint factorizes:

$$P(\text{words} \mid \text{class} = k) = \prod_t P(t \mid k)^{x_t}.$$

- Clearly false (“interest” and “rate” travel together) — but it turns the model into a product of one-word probabilities we *can* estimate.
- Still a strong baseline: for *ranking* classes the errors often wash out.

Naive Bayes: Estimation and Prediction

Back to the framework. The **model** parameters are $\mu_{kj} = P(\text{word } j \mid \text{class } k)$; the **loss** is the negative log-likelihood. Its closed-form minimizer is the within-class word frequency, with the pseudo-count c as **regularization** (a prior, so an unseen word does not zero out a class):

$$\widehat{\mu}_{kj} = \frac{c + \sum_i \pi_{ik} x_{ij}}{Jc + \sum_i \sum_j \pi_{ik} x_{ij}}$$

Predict the **most likely class**, adding logs instead of multiplying:

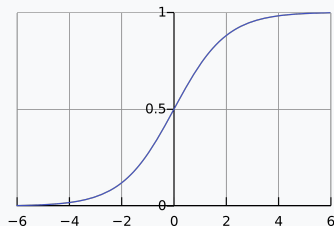
$$\hat{y}_i = \arg \max_k \left[\log \hat{\alpha}_k + \sum_j x_{ij} \log \widehat{\mu}_{kj} \right].$$

x_{ij} : count of word j in doc i ; $\pi_{ik} = 1$ if doc i is class k ; J : vocabulary size; $\hat{\alpha}_k$: class prior. The Jc keeps $\sum_j \widehat{\mu}_{kj} = 1$.

Logistic Regression: the Model

A **discriminative** alternative: model the class probability directly as a function of a linear index $x'\beta$. Map the index into $[0, 1]$ with the **logistic (sigmoid)** function:

$$\hat{y} = \sigma(x'\hat{\beta}) = \frac{1}{1 + \exp(-x'\hat{\beta})}.$$



The sigmoid keeps the output a valid probability; predict class 1 when $\hat{y} \geq 0.5$. Each $\hat{\beta}_j$ is a signed weight on word j .

Logistic Regression: Loss and Fitting

Loss — cross-entropy (log loss), the negative log-likelihood economists know from MLE:

$$L(\beta) = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right].$$

Fitting — no closed form, but the loss is **convex**, so the gradient descent from the OLS slide finds the global optimum. The gradient has the same clean form:

$$\frac{\partial L}{\partial \beta_j} = (\sigma(x_i' \beta) - y_i) x_{ij}.$$

Same four ingredients as OLS and Naive Bayes — only f and L changed.

Reading a Linear Classifier: a Learned Dictionary

Because logistic regression is linear, its fitted coefficients $\hat{\beta}_j$ are directly readable: sort them and look at the two extremes.

Largest positive $\hat{\beta}_j$ (5★)

great, wonderful, best,
excellent, highly

Largest negative $\hat{\beta}_j$ (1★)

boring, worst, waste,
disappointed, bad

Illustrative of what the Class-2 notebook prints on the book reviews. This recovers *from data* the kind of word list we wrote by hand in Class 1 — the same object as Gentzkow–Shapiro discriminating words, and as the partisan bigrams we will meet in Kempf & Cassidy. A sanity check *and* a result.

Regularization: a Knob, Not a New Model

Any linear model overfits when features (words) outnumber documents. Add a **penalty** on coefficient size to trade fit for generalization:

$$\min_{\beta} \sum_i L(y_i, x_i' \beta) + \lambda \|\beta\|, \quad \ell_2 \text{ (ridge)}, \ell_1 \text{ (lasso)}.$$

- Ridge (ℓ_2) shrinks all coefficients; lasso (ℓ_1) zeros some out — automatic feature selection.
- λ (or $C = 1/\lambda$) is a **hyperparameter**, tuned by cross-validation: high $C \Rightarrow$ train accuracy $\rightarrow 1.0$ while test flattens — watch the gap.

The *same* penalty applies to OLS, logistic, and the multinomial logit — which is exactly the engine of the second use of ML, next.

Measuring Group Differences

Here the goal is not to label documents but to measure **how different two groups are** — the model's *power to separate them* is the estimand.

- **Gentzkow, Shapiro & Taddy (2019)**: how distinguishable is Republican from Democratic congressional speech? The size of that gap is a measure of **partisanship**.
- **The catch**: with tens of thousands of phrases the naive measure is biased *upward* — rare phrases look “partisan” by chance, so *any* two groups seem easy to separate.
- **The fix**: estimate the differences with a **penalized multinomial logit**, shrinking rare-phrase effects down to an honest measure.

Same penalized model as the previous slide, now read for the *size of the gap* rather than per-document labels. The generated-variable problem reappears — here in the **difference estimate itself** (Class 5).

Why Lasso Is Often Too Limited for Text

Penalized *linear* models are the workhorse for tabular economic data. For **text** they are often too limited — and it is worth seeing why:

- **Additivity.** A linear model sums per-word contributions. But “not good” is not good + not; meaning lives in **combinations**.
- **Dependency structure.** Term counts are strongly correlated — topics pull whole groups of words together — and lasso ignores this (Ash & Hansen 2023).
- **Linearity.** A word’s effect can be non-monotone or context-dependent; a straight line in $x'\beta$ cannot bend.

⇒ We want models that capture **interactions and nonlinearities** among word counts.

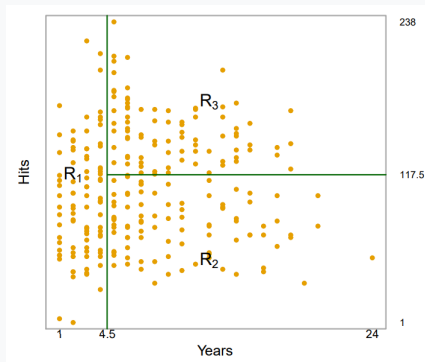
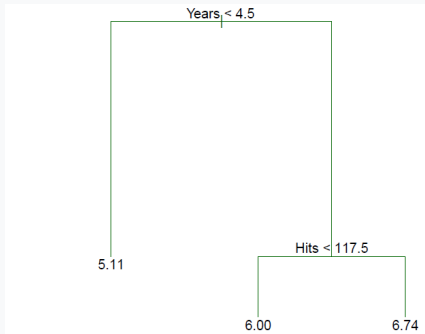
Tree-Based Models: the Basic Assumption

- **Linear models** assume an additive form
$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$
- **Tree-based models** instead **partition the feature space** into regions:
 - each document falls into one region;
 - the prediction is whatever that region says.

This is exactly what lets them capture the **nonlinearities and interactions** among word counts that lasso misses.

Main reference: An Introduction to Statistical Learning, Ch. 8.

A Single Tree, and Its Partition



A tree is a sequence of yes/no questions; the leaves are the regions of the partition, each with its own prediction.

Building a Tree: Recursive Binary Splitting

1. Start with all data in a single node.
2. For each feature, find the split point that minimizes the loss function:
 - Regression: Sum of Squared Residuals (SSR);
 - Classification: entropy.
3. Choose the feature and split point with the lowest loss.
4. Create the child nodes from the split.
5. Recursively repeat steps 2–4 on each child node.
6. Stop at a stopping criterion (e.g. max depth, min samples per leaf).

Greedy and fast. A single deep tree **overfits** — which is why we rarely stop here. Gini impurity is the usual alternative to entropy.

From One Tree to an Ensemble

Combine many trees to cut variance:

- **Bagging** — grow trees on bootstrap samples, average them.
- **Random forests** — bagging + a random feature subset at each split; further decorrelates the trees.
- **Boosting** — grow shallow trees sequentially, each fit to the previous model's residuals.

Ash & Hansen (2023) for count features: random forests and gradient boosting are more robust than lasso for text — and on *long* documents can even beat Transformers.

Generative vs. Discriminative

Two philosophies, now side by side:

- **Generative** (Naive Bayes): model how each class *produces* words, $p(\mathbf{x}, y)$, then invert to get $p(y | \mathbf{x})$.
- **Discriminative** (logistic, trees): model the class *boundary* $p(y | \mathbf{x})$ directly, never describing how words are generated.

The discriminative model solves the easier problem and has **lower error in the limit** — but it needs more data to get there.

Which Rung? Sample Size and the Task

- Ng & Jordan (2001): the generative model has higher asymptotic error but **converges faster** — it reaches its (worse) floor with far fewer labels.
- Economics corpora often have only a *few hundred* hand-labels (vs. millions in CS, Ash & Hansen 2023). With small N , the extra structure of a generative model can win.

So: start simple and match the rung to the task and the label budget, not to fashion. Text-specific generative models include supervised LDA and multinomial inverse regression (Taddy 2013, 2015) — which we meet again in Kempf & Cassidy.

Evaluating a Classifier

Always Start With a Baseline

Before celebrating an accuracy number, ask: **what would a trivial rule get?**

- The **majority-class baseline** always predicts the most common label. A useful model must beat it.
- Under imbalance the baseline is brutal: if 95% of documents are one class, predicting that class blindly already scores 95% — and is useless.
- On balanced 1★/5★ book reviews (10,000 reviews): logistic ≈ 0.85 , Naive Bayes ≈ 0.83 , vs. baseline 0.50. Here the gain over baseline is real.

This is why a single accuracy figure, with no baseline, says almost nothing.

The Confusion Matrix

	Predicted	
	Positive	Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Every classification metric is a ratio of these four cells. Read it before trusting a single number.

Accuracy

	Predicted	
	Positive	Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

How often we are right on average — decent *only* when classes are balanced.

Precision

	Predicted	
	Positive	Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

$$\text{Precision} = \frac{TP}{TP + FP}$$

Of the documents *flagged* positive, how many truly are. “Can I trust a positive prediction?”

Recall

	Predicted	
	Positive	Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

$$\text{Recall} = \frac{TP}{TP + FN}$$

Of the truly positive documents, how many we *caught*. “Did I miss many?”

F1 and Balanced Accuracy

Precision and recall trade off — summarize both:

$$F1 = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right)$$

Under imbalance these are far more honest than raw accuracy. With K classes, average per-class metrics (optionally frequency-weighted).

An Applied Case: Kempf & Cassidy (2025)

“Partisan Corporate Speech” — do U.S. firms increasingly talk like partisan politicians? The whole Class-2 toolkit shows up in one paper.

- **Core idea (Gentzkow–Shapiro logic):** learn what partisan language looks like from a **labeled benchmark** — Republican vs. Democratic *politicians’* tweets — then **scale** the measure to S&P 500 *corporate* tweets, 2012–2022.
- The output is a **Partisan Speech Index (PSI)**: the posterior probability that a tweet “sounds” Republican vs. Democratic.

Kempf & Cassidy: the Model Is One of Ours

The PSI is built with **Multinomial Inverse Regression (MNIR)** — Taddy (2013, 2015), as used by Gentzkow, Shapiro & Taddy (2019):

- bigram counts are drawn from a **multinomial** whose probabilities depend on the speaker's party — the penalized generative count model from our regularization and generative-vs-discriminative slides;
- the heavily weighted bigrams form a **learned dictionary** of partisan phrases — exactly the read-the-coefficients move;
- it also **shrinks** rare phrases and lets the partisan vocabulary **vary over time** — avoiding the look-ahead bias of a pretrained LLM.

Kempf & Cassidy: the Rest of the Toolkit

Two more pieces, each a method from this course:

- **A fine-tuned Transformer** (RoBERTa, Class 4) flags “action” tweets — concrete commitments. Trained on 9,268 hand-labeled tweets, it reaches **precision 0.90, recall 0.95, accuracy 0.985** — the metrics from the last few slides, in the wild.
- **A topic model** (with ChatGPT used to label topics) organizes partisan tweets into themes — DEI and climate on one side, economy and energy on the other — our unsupervised half.

Validation: partisan tweets draw sharper reactions from politicians — the measure tracks something real. Label, train, scale, interpret, validate.

Scale the Measure — Then Validate It

Scale. The trained model labels the *entire* corpus at near-zero cost — the payoff of supervised learning. Aggregate the per-document scores to the unit you study: average $\hat{P}(\text{positive})$ by author, firm, or year.

Validate. The measure is only as good as the model:

- **hand-check the most uncertain cases** ($\hat{p} \approx 0.5$) — negation, mixed opinions, off-topic text — where errors concentrate;
- the output is a **generated variable** with error; Class 5 formalizes the consequences for inference.

A Warning: Predicting on the Wrong Signal

A classifier will use *whatever* correlates with the label in the training data — not necessarily the concept you mean.

- To predict right-wing ideology it may lean on “**Texas**”; to predict remote work, on “**software**.”
- These are **correlates**, not the construct. On the training corpus they help accuracy — so cross-validation will *not* flag them.

Fine for pure prediction. Dangerous the moment the predicted label becomes a **variable in a regression**.

Why It Biases Inference — and How to Check

Suppose you regress an outcome on the predicted label, and your treatment (say, a policy) **also shifts the correlate** (more tech firms $\rightarrow$ more “software”).

- Then the predicted label moves with the treatment *even when the true concept does not* — an **exclusion-restriction violation**, hence bias.
- More flexible models (boosting, BERT) are *more* exposed: they exploit subtler style cues.

Validation test: on a hand-labeled sample, show the model’s **error is uncorrelated with the treatment**. Class 5 formalizes what to do when it is not.

Unsupervised Discovery

Unsupervised Learning as Dimensionality Reduction

With **no labels**, the goal is to compress the huge bag-of-words — thousands of terms — into a **few interpretable dimensions**, then ask “*what is in this corpus?*” Three flavours:

- **Clustering** — each document into one of k groups;
- **Topic models** — each document a *mixture* of a few topics;
- **PCA / LSA** — each document a few continuous components.

All three are **dimensionality reduction**. Powerful to **explore**, fragile as **measurement** (the output shifts with the method, k , and the seed). First: a notion of **closeness**.

The Vector Space Model

Documents as Vectors

Everything here starts from one picture: a document is a **vector** in feature space — a row of the document-feature matrix from Class 1.

The simplest similarity is the **inner product** of two (0/1) document vectors:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + \cdots + a_J b_J = \# \text{ words the two documents share.}$$

Counts add information over presence/absence — but then a long document scores high with everything, simply for being long.

Cosine Similarity

Divide out length: compare **direction**, not magnitude. With the ℓ_2 norm $\|\mathbf{x}\| = \sqrt{\mathbf{x} \cdot \mathbf{x}}$,

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \quad \begin{cases} 1 & \text{same direction} \\ 0 & \text{orthogonal (no shared words)} \\ -1 & \text{opposite} \end{cases}$$

- Magnitude-free, so document length drops out.
- The workhorse similarity for text — we use it again on *word* vectors in Class 3.

From Similarity to Distance

Many algorithms — including k -means — need a **distance**, not a similarity:

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_j |x_j - y_j|^p \right)^{1/p}, \quad \begin{cases} p = 1 & \text{Manhattan} \\ p = 2 & \text{Euclidean} \end{cases}$$

- On length-normalized vectors, Euclidean distance and cosine rank neighbors almost identically.
- How you represent the document (counts, tf-idf, later embeddings) changes every distance — so it is a modelling choice, not a detail.

Clustering

K-means Clustering

Partition documents into k groups by distance between their (tf-idf) vectors:

1. Initialize k centroids.
2. Assign each document to the nearest centroid.
3. Recompute centroids as cluster means; repeat to convergence.

It minimizes within-cluster squared distance and gives a **hard assignment** — each document lands in exactly one group. Works on *any* vector representation, including the embeddings of Class 3.

Choosing k

No labeled answer for the number of clusters:

- Use the **silhouette** score and the **elbow** in inertia as guides.
- Read each cluster's centroid terms.
- The final call is a judgment about **interpretability**, not a number.

Results are sensitive to k , the metric, and the representation — **validate** by reading representative documents in each cluster.

Dimensionality Reduction

The Curse of Dimensionality

In the bag-of-words model, each term is a dimension — often **tens of thousands**.

- expensive computation; hard to visualize; easy to overfit
- the count vectors are **sparse**, so raw distances are noisy

Can a *few* directions capture most of the signal?

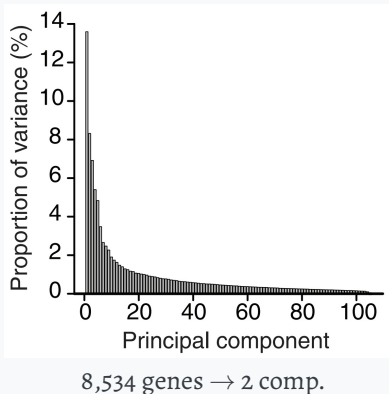
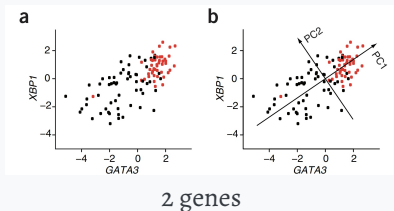
PCA / Latent Semantic Analysis

Familiar to economists as factor analysis: find orthogonal directions of maximum variance. Applied to the document-term matrix, PCA is called **LSA**.

$$X \approx U_k \Sigma_k V_k^\top$$

- Loadings on each component are interpretable latent themes.
- The $K \ll V$ reduced representation speeds up downstream models.
- On sparse counts, truncated SVD / NMF are the practical variants.

PCA: Two Genes vs. Many



The same compression that helps with genes helps with words.
Source: Ringnér (2008).

Topic Models: a Pointer

The other classic unsupervised tool is the **topic model** (LDA; Blei, Ng & Jordan 2003): each document is a *mixture* of latent topics, each topic a distribution over words.

- It was **the** text-as-data workhorse of the 2010s — e.g. Hansen, McMahon & Prat (2018) on FOMC transcripts.
- Today it is a **baseline**: embedding-based clustering (Class 3) usually gives more coherent themes.

The full generative model, a worked example, and inference are in the **appendix** — worth knowing for reading the literature.

Why Discovery Is Not Measurement

- Clusters/topics move with k , seed, and preprocessing.
- They answer “what themes exist,” not “how much of construct X is in document d .”
- The number that maximizes fit rarely maximizes *interpretability*.

Excellent to **explore**; weak to **measure** an economic concept without heavy human validation.

Where LLMs Fit

The Performance–Interpretability Frontier

Interpretable, lower accuracy

- dictionaries
- logistic / Naive Bayes

High accuracy, opaque

- tree ensembles
- fine-tuned Transformers (BERT)

No model dominates on *both* axes. The right choice depends on whether you need a mechanism or a number (Ash & Hansen 2023).

So Is Today's Toolkit Obsolete? No.

When an LLM predicts, it sits at the opaque end — and three things keep today's models in play:

- **Interpretability** — when you want the mechanism, not just the label.
- **Long documents** — boosting on counts can beat Transformers.
- **Inference** — the generated-variable problem gets *worse*, not better, with subtler models.

And the **workflow is unchanged**: in Class 4 we swap the labeler for an LLM and reuse every evaluation and validation step from today.

Takeaways

1. ML with text is a **measurement workflow**: label, train, evaluate *out of sample*, scale, validate.
2. Climb the model ladder by task — lasso is often too limited for text; ensembles and (later) Transformers do more.
3. The output is a **generated variable** with error — hand-check borderline cases; beware confounded predictors.
4. Unsupervised = discovery, not measurement: represent documents as vectors, cluster, and validate everything.
5. Topic models still work but are now a baseline (LDA in the appendix); embeddings (Class 3) often do better.

Live: Class 2 notebook. **Practice:** Problem Set 2.

Appendix: LDA Topic Models

LDA Topic Models

Each document is a **mixture of latent topics**; each topic a distribution over words (Blei, Ng & Jordan 2003).

$$\theta_d \sim \text{Dir}(\alpha), \quad \phi_k \sim \text{Dir}(\beta), \quad z_{d,n} \sim \theta_d, \quad w_{d,n} \sim \phi_{z_{d,n}}$$

A Bayesian factor model for *discrete* data: the principled cousin of LSA.

Clustering vs. Topic Models

Clustering:

- k groups; each document belongs to *one*.

Topic models:

- each document is a *mixture* of topics (potentially all of them);
- topics are distributions over words the researcher then interprets.

The Generative Process

Imagine producing the corpus one word at a time:

- N documents, M words each; J tokens, K topics.
1. For each document i : draw a topic mixture $\boldsymbol{\pi}_i \sim \text{Dirichlet}(\boldsymbol{\alpha})$.
 2. For each word m in document i :
 - 2.1. draw a topic $Z_{im} \sim \text{Multinomial}(1, \boldsymbol{\pi}_i)$;
 - 2.2. draw a word $\tilde{W}_{im} \sim \text{Multinomial}(1, \boldsymbol{\mu}_{Z_{im}})$.
 3. Aggregate into the document-term matrix W .

Worked Example: *Sports & Animals Illustrated*

A hypothetical magazine:

- $K = 2$ topics: Sports and Animals
- $J = 8$ vocabulary words: *Team, Horse, Score, Run, Habitat, Predators, Training, Wildlife*
- $M = 4$ words per article

Let us run the generative process by hand.

Parameters: Topic Weights π

Document-level mixtures over the two topics:

	Sports	Animals
Doc 1	0.8	0.2
Doc 2	0.1	0.9
Doc 3	0.5	0.5

Parameters: Word Probabilities μ

Each topic is a distribution over the eight words:

	Team	Horse	Score	Run	Habitat	Predators	Training	Wildlife
Sports	.15	.1	.35	.2	0	0	.2	0
Animals	0	.2	0	.15	.15	.25	0	.25

Draw Three Articles

For each word: draw a topic from π_i , then a word from μ of that topic.

Article 1 (Sports-heavy): Team, Score, Run, Training

Article 2 (Animals-heavy): Horse, Habitat, Predators, Wildlife

Article 3 (mixed): Run, Training, Horse, Habitat

The Resulting Bag-of-Words Matrix

	Team	Horse	Score	Run	Habitat	Predators	Training	Wildlife
Article 1	1	0	1	1	0	0	1	0
Article 2	0	1	0	0	1	1	0	1
Article 3	0	1	0	1	1	0	1	0

This W is all we observe. **Inference runs the process backward:** recover π and μ from W .

Inference in LDA

We estimate the posterior of:

- π — topic mixtures for each document;
- μ — word distributions for each topic.

Compare the observed word counts against what the model and prior expect — the same logic as before, but the exact posterior is intractable, so we approximate it.

Applying LDA in Practice

1. **Choose** K from metrics or domain knowledge.
2. **Train**: obtain topic-word and document-topic distributions; pull representative documents; **label topics by hand**.
3. **Use** document-topic shares as features in downstream analysis.

Hansen, McMahon & Prat (2018) use LDA on FOMC transcripts; topic shares respond to a transparency reform.

A Validated Application

TABLE 1 A Summary of Common Assumptions and Relative Costs Across Different Methods of Discrete Text Categorization

	Method				
	<i>Reading</i>	<i>Human Coding</i>	<i>Dictionaries</i>	<i>Supervised Learning</i>	<i>Topic Model</i>
A. Assumptions					
<i>Categories are known</i>	No	Yes	Yes	Yes	No
<i>Category nesting, if any, is known</i>	No	Yes	Yes	Yes	No
<i>Relevant text features are known</i>	No	No	Yes	Yes	Yes
<i>Mapping is known</i>	No	No	Yes	No	No
<i>Coding can be automated</i>	No	No	Yes	Yes	Yes
B. Costs					
Preanalysis Costs					
<i>Person-hours spent conceptualizing</i>	Low	High	High	High	Low
<i>Level of substantive knowledge</i>	Moderate/High	High	High	High	Low
Analysis Costs					
<i>Person hours spent per text</i>	High	High	Low	Low	Low
<i>Level of substantive knowledge</i>	Moderate/High	Moderate	Low	Low	Low
Postanalysis Costs					
<i>Person-hours spent interpreting</i>	High	Low	Low	Low	Moderate
<i>Level of substantive knowledge</i>	High	High	High	High	High

Quinn, Monroe, Colaresi, Crespin & Radev (2010): a carefully *validated* topic model of political attention.

Validate Topics by Hand

A topic model gives word lists — not their meaning or coherence.

1. Pull the documents that load most heavily on a topic.
2. Read them.
3. Judge coherence; rate it.

Formal versions (intruder-word tests) ask annotators to spot a word that does not belong. Same move as for dictionaries and classifiers.

LDA in 2026: a Baseline

The classic text-as-data tool of the 2010s — still useful, but now **a baseline**.

- Embedding-based methods (Class 3) usually give more coherent themes and work on any vector representation.
- Reach for LDA as a transparent first pass or filter; expect to compare it against embeddings.